CH-01

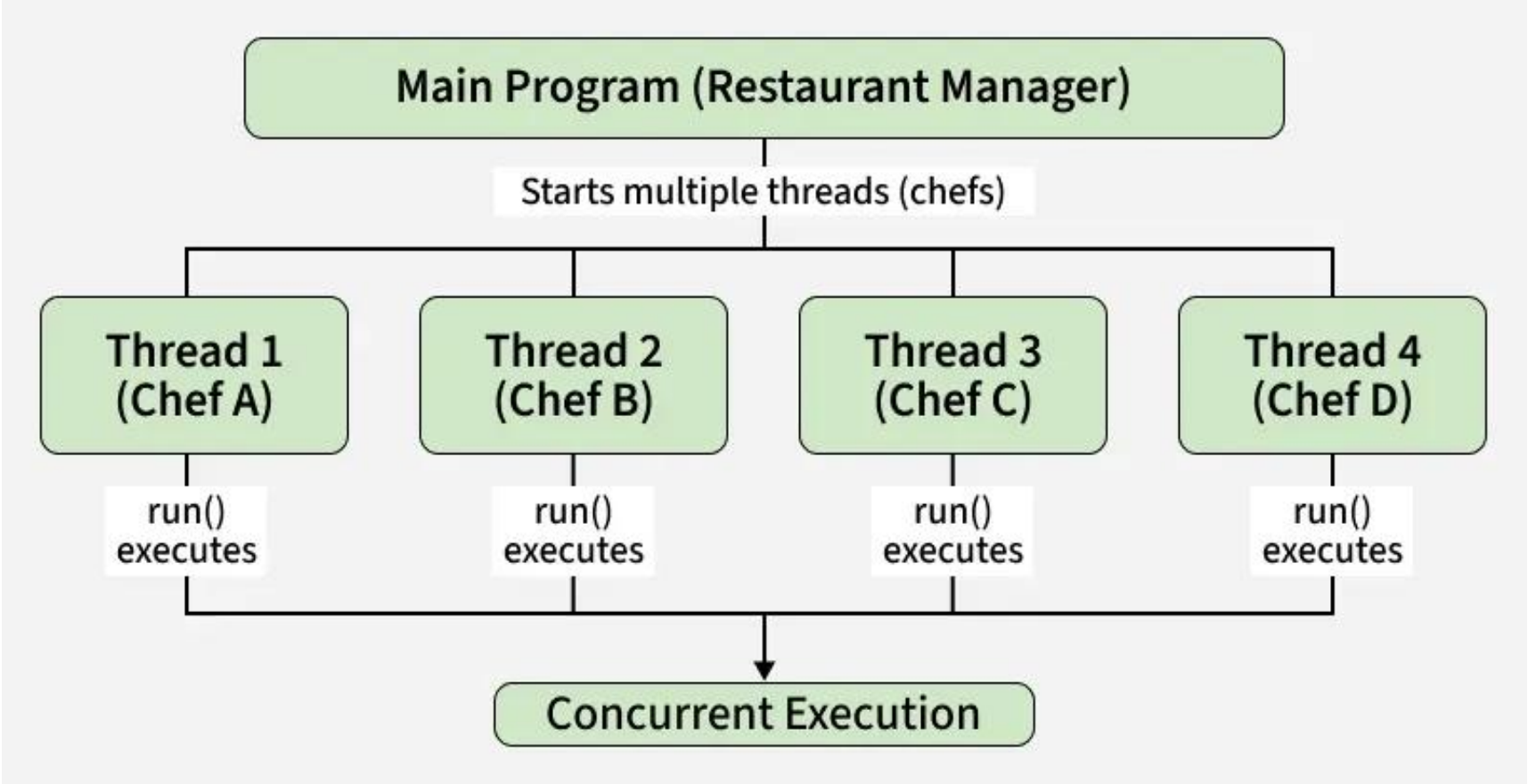
Multithreaded Programming

Multithreading in Java

- Multithreading in Java is a feature that enables a program to run multiple threads simultaneously, allowing tasks to execute in parallel and utilize the CPU more efficiently.
- A thread is a lightweight, independent unit of execution inside a program (process).
- A process can have multiple threads.
- Each thread runs independently but shares the same memory.

Java Thread Model

- **Example:** Imagine a restaurant kitchen. Multiple chefs (threads) are preparing different dishes at the same time. This speeds up service and utilizes all available resources (CPU).



- Threads can be created by using two mechanisms:

1. Extending the Thread class with Syntax

- We create a class that extends Thread and override its run() method to define the task. Then, we make an object of this class and call start(), which automatically calls run() and begins the thread's execution.
- public class Main extends Thread {

```
    public void run() {
        System.out.println("This code is running in a thread"); } }
```

2. Implementing the Runnable Interface with Syntax

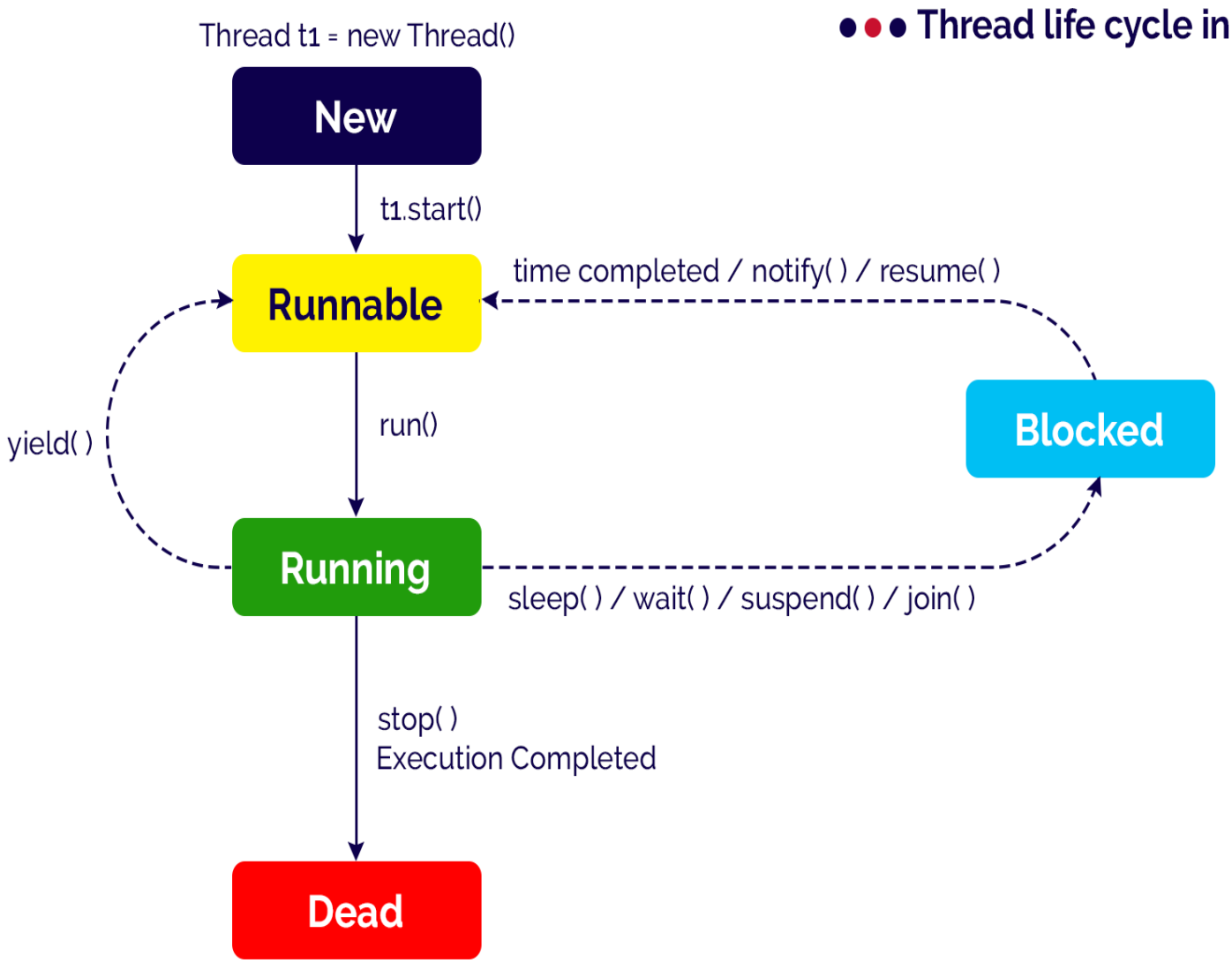
- We create a new class which implements java.lang.Runnable interface and define the run() method there. Then we instantiate a Thread object and call start() method on this object.
- public class Main implements Runnable {

```
    public void run() {
        System.out.println("This code is running in a thread");
} }
```

- **Implement Example**
- If the class implements the Runnable interface, the thread can be run by passing an instance of the class to a Thread object's constructor and then calling the thread's start() method:
- public class Main implements Runnable {

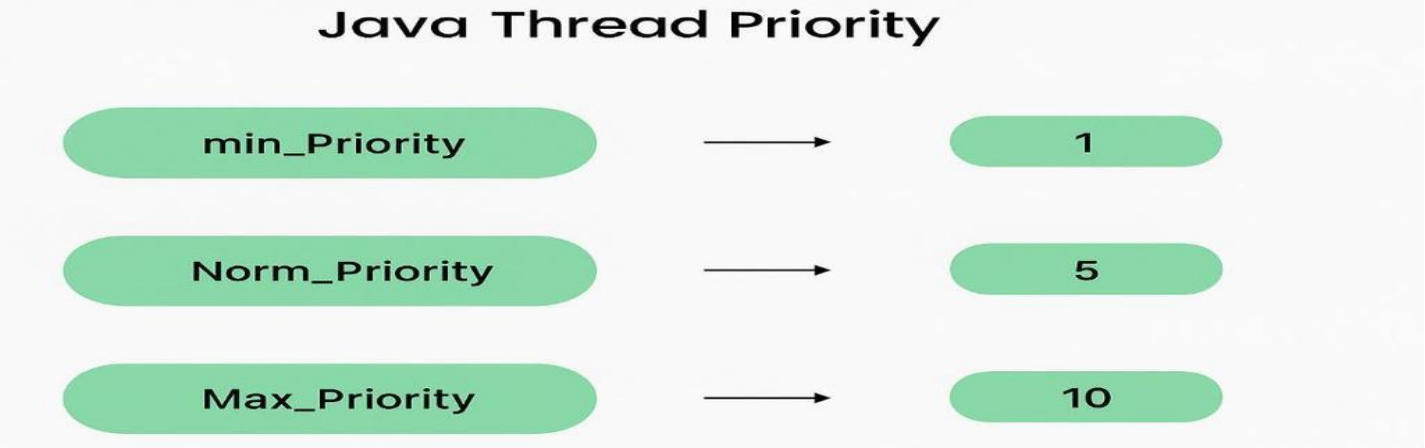
```
public static void main(String[] args) {
 Main obj = new Main(); Thread thread = new Thread(obj);
 thread.start();
System.out.println("This code is outside of the thread"); } public void run() {
System.out.println("This code is running in a thread"); } }
```

- **Thread Life Cycle in Java**



Java Thread Priority in Multithreading

- Java allows multiple threads to run concurrently, and the Thread Scheduler decides the order of execution.
- Each thread has a priority between 1 and 10, which indicates its importance.
- Threads with higher priority are generally preferred by the scheduler, though execution order is not guaranteed.



wait(), notify() and notifyAll() methods in Java

- The threads can communicate with each other through **wait()**, **notify()**, and **notifyAll()** methods in Java.
- These are the final methods defined in the Object class and can be called only from within a synchronized context.
- The wait() method causes the current thread to wait until another thread invokes the notify() or notifyAll() methods for that object.
- These methods will throw an **IllegalMonitorStateException** if the current thread is not the owner of the object's monitor.

The wait() Method

- In Java, the **wait()** method of the Object class allows threads to pause their execution and wait for a specific condition to be satisfied or until the **notify()** or **notifyAll()** method is invoked by some other thread of this object.
- Following is the syntax of the **wait()** method:
- public final void wait() throws InterruptedException

The notify() and notifyAll() Method

- The **notify()** method of the Object class is used to wake up a single thread that is waiting on this object's monitor. Following is the syntax of the **notify()** method:
- public final void notify()
-

  The **notifyAll()** method of the Object class is used to wake up all threads that are waiting on this object's monitor. A thread waits on an object's monitor by calling the object's **wait()** method.
- Following is the syntax of the **notifyAll()** method:
- public final void notifyAll()